

StockPilot — System Guide

A printable plain-English walkthrough of the entire inventory system, written for someone who isn't a developer but wants to understand how their own product works.

Last updated: 2026-05-19

How to read this

The document moves from very high to very specific. Read top-to-bottom the first time; later you can skip to any one section. Every time it mentions a file path (like `apps/web/src/app/...`), that's a real file you can find in the repo and open in VS Code or GitHub.

Bold-italic terms appear in the glossary at the back.

Chapter 1 — The thirty-second summary

StockPilot is a private inventory management system used inside one organization (currently L4L Fresno, a charter school). It tracks every physical thing the org owns — books in the textbook room, t-shirts in the swag closet, canopies in the events shed — and every movement of those things (received from a supplier, sent to a school site, rented to an employee for an event, counted during inventory audit, etc).

The system has five surfaces:

1. **The web dashboard** — what staff and managers use from their computer. This is the bulk of the app.
2. **The mobile app** — for in-warehouse scanning, AI shelf-counting, and on-the-floor checkout.
3. **The public order link** — a single URL teachers/partners use to request books and supplies from the warehouse without needing an account.
4. **Email** — order receipts, signature confirmations, weekly digests.
5. **The Supabase database** — the underlying source of truth that all of the above read from and write to.

Everything runs in the cloud. There are no physical servers you have to maintain.

Chapter 2 — What StockPilot does, in business terms

Forget code for a second. The system answers business questions like:

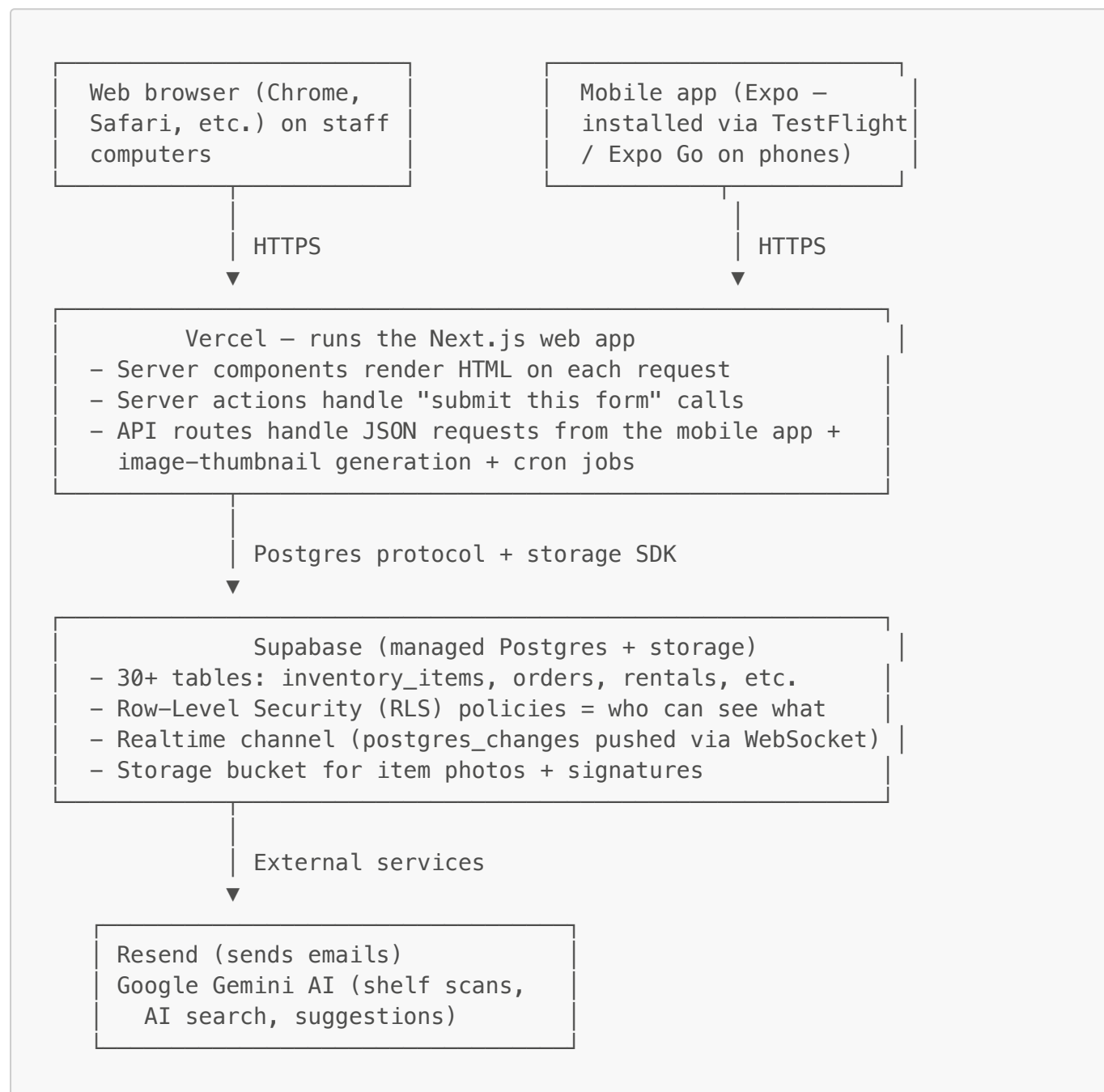
- *"How many size-XL Stronger Than Ever t-shirts are in DC4 right now?"* → Inventory list, filtered by warehouse + category + size.
- *"Mr. Hernandez ordered 30 World History textbooks for his class. Where in that workflow are we?"* → Orders detail page, status pill at the top.
- *"Who currently has the canopy?"* → Rentals page, filter Out.
- *"What's the total \$ value of our inventory across all four warehouses?"* → Reports → Inventory Valuation.

- "Did Karen actually return the projector she signed out last Friday?" → Rentals page, search by borrower name.
- "Someone scanned a barcode on the floor. What is it?" → Mobile app, scanner mode.

Every page in the dashboard exists to answer one or two such questions fast. When a new question keeps coming up that the existing pages can't answer, that's when we add a new page or feature.

Chapter 3 — The big-picture architecture

This is the most important diagram in the doc. If you remember nothing else, remember this shape.



Five things to internalize from this diagram:

1. **The browser doesn't talk directly to the database.** It always goes through Vercel-hosted Next.js code. That's important because the Next.js layer is where we enforce business rules and security before the DB is touched.

2. **Vercel is "the server."** When you visit stockpilotusa.com/dashboard/inventory, your browser asks Vercel for the page. Vercel runs a small TypeScript program that queries Supabase, formats the answer into HTML, and ships it back. There's no "server" you have to log into and maintain — Vercel does all of that.
3. **Supabase is the only place data actually lives.** If we lose Vercel, we lose nothing important — we just redeploy. If we lose Supabase, we lose everything. (That's why we have nightly backups and never run destructive SQL without a migration file.)
4. **The mobile app is mostly UI.** It calls the same JSON endpoints on Vercel that the browser uses. There's no separate "mobile server."
5. **Outside services do one job each.** Resend = email. Google Gemini = AI. They're plugged in via API keys and we could swap them for competitors without rewriting business logic.

Chapter 4 — The technology stack, plain English

Technology	What it is	What it does for us
Next.js 16	A framework for building web apps in TypeScript	Powers everything you see in your browser when you visit stockpilotusa.com . Handles routing (URLs to pages), rendering (HTML generation), and forms.
React 19	A library for building user interfaces	The actual buttons, inputs, lists, cards on every page are written as "React components." Next.js builds on top of React.
TypeScript	JavaScript with types	A type-safe version of JavaScript. The compiler catches typos and wrong-type bugs before users do. Worth its weight in gold.
Tailwind CSS	A styling system	Instead of writing CSS files, every visual property (spacing, color, font) gets a short class name like <code>bg-card p-4 rounded-lg</code> . Fast iteration.
shadcn/ui	A library of UI building blocks	Pre-built buttons, dropdowns, dialogs, etc. that look professional out of the box. Sits on top of Tailwind.
Supabase	A hosted database service	Wraps a PostgreSQL database + file storage + realtime + auth into one product. Like Firebase but Postgres-based.
PostgreSQL	An industrial-strength database	The actual database engine behind Supabase. Most banks and big SaaS products run on it.
Row-Level Security (RLS)	A Postgres feature	Lets us write security rules ON the database tables themselves so the database refuses to return rows the user shouldn't see. We rely on this heavily.
Vercel	A hosting platform	Runs our Next.js code. Every <code>git push</code> to main triggers an automatic deployment.

Technology	What it is	What it does for us
Resend	An email-sending service	Delivers our transactional emails (order receipts, signature confirmations, weekly digests).
Google Gemini 2.0 Flash	An AI model	Powers the AI shelf-scan (count books from a photo) and the natural-language AI assistant in /dashboard/ai .
Expo + React Native	A mobile-app framework	Lets us write the iOS/Android app in the same language (TypeScript) as the web. The mobile app lives in apps/mobile/ .
Turborepo + pnpm	Tools for managing a monorepo	We keep web, mobile, and shared code in ONE repository. These tools coordinate building/testing across them.
vitest	A test runner	Runs the automated tests. Every business rule has tests so we don't regress when changing code.

You don't have to memorize this. The point is to recognize the names when they come up in commits and have a rough sense of "this thing is responsible for X."

Chapter 5 — How the code is laid out

The whole project lives in [/Users/brandenvincent-walker/Desktop/Inventory System App/](#).
Top-level:

```

.
├── apps/
│   ├── web/           ← The web dashboard (Next.js app – 90% of the code)
│   └── mobile/        ← The phone app (Expo)
├── packages/
│   └── core/          ← Shared TypeScript: types, zod schemas, permission
rules
├──   ├── config/      ← Shared ESLint/TypeScript config
│   └── db/            ← Database type definitions (mostly stubbed)
├── supabase/
│   └── migrations/    ← Every database change ever made. 128 files as of
today.
├── docs/              ← Documentation. THIS file lives in here.
├── scripts/           ← Small dev/maintenance scripts
├── BLUEPRINT.md       ← Original product plan
└── package.json       ← Top-level project metadata

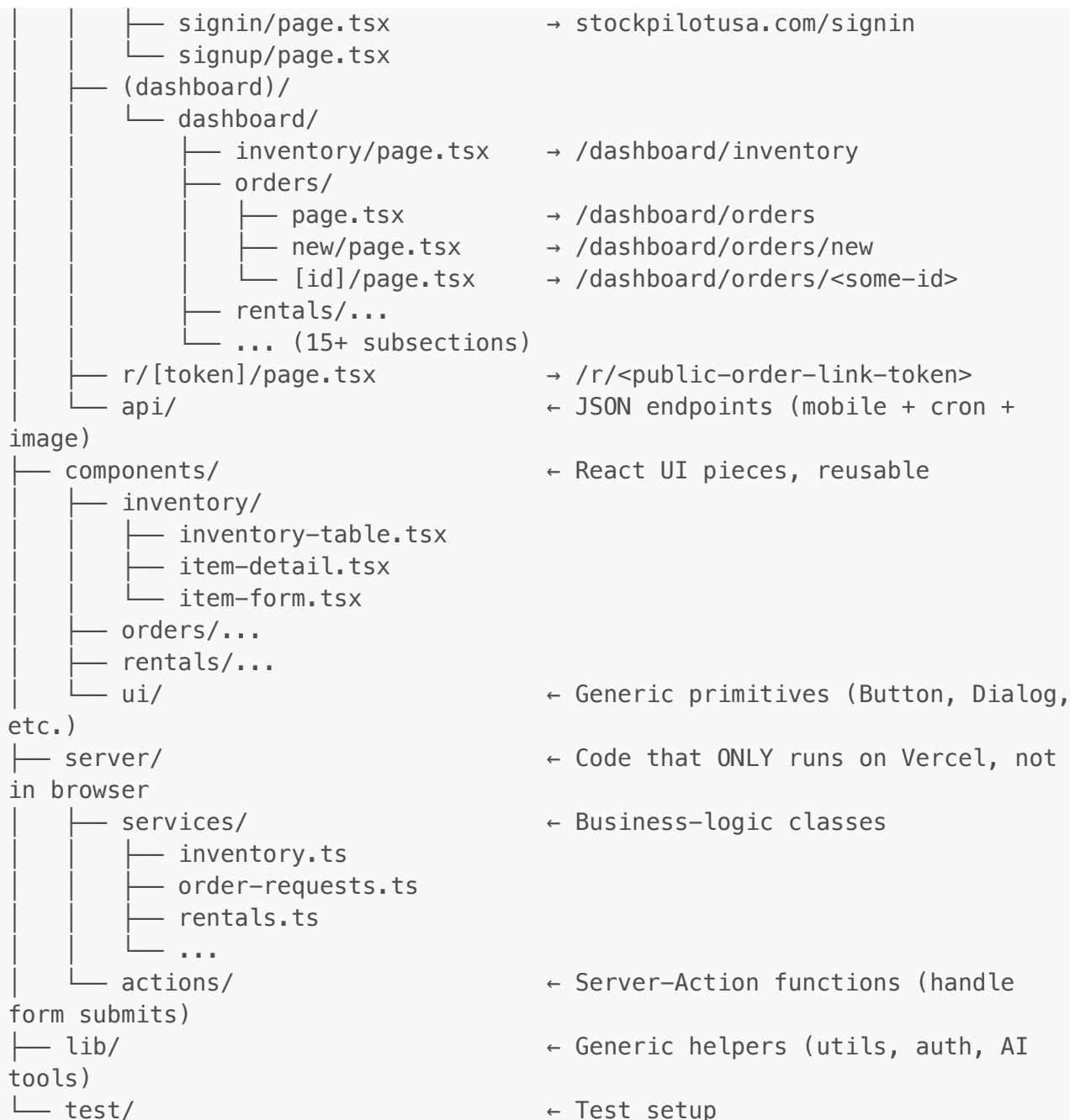
```

The web app is the busiest. Zooming in:

```

apps/web/src/
├── app/                ← Next.js routes – every folder = a
URL
└── (auth)/

```



One folder = one concern. When you're hunting for "where does the Mark Returned button on a rental do its work?" the trail goes:

1. The button is rendered in `components/rentals/rental-actions-panel.tsx`
2. Clicking it calls a function in `server/actions/rentals.ts`
3. That function uses `server/services/rentals.ts` to do the actual DB work
4. That writes to the `rentals` table (created in `supabase/migrations/0131_rentals.sql`)

This four-layer pattern — **page → action → service → database** — repeats for nearly every feature.

Chapter 6 — End-to-end: how one button click actually works

Let's trace a real example all the way through. Say a teacher is on the public order link and clicks **Send request** with a book in their cart.

Step 1 — In their browser

They're on the page at `apps/web/src/app/r/[token]/page.tsx`. That page rendered earlier when they loaded the URL — server-side on Vercel — and shipped them HTML + a small bundle of React JavaScript.

When they click Send, the React component (`apps/web/src/components/orders/public-v2/public-cart-rail.tsx`) runs a function called `handleSubmit`. That function:

- Collects the cart state (which items + how many)
- Collects the form fields (name, email, phone, delivery site)
- Calls `fetch('/api/v1/public/order-requests', { method: 'POST', body: JSON.stringify(...) })`

Step 2 — Hits a Vercel function

`/api/v1/public/order-requests` maps to the file `apps/web/src/app/api/v1/public/order-requests/route.ts`. Vercel sees the POST request, runs that file's `POST` function with the request body.

That file:

1. **Validates the input** using a zod schema (a tool that defines "what fields should this look like"). Reject if the email isn't valid, the token is malformed, the lines are empty, etc.
2. **Checks the honeypot field** (an anti-bot trick — see Chapter 13).
3. **Checks rate limits** (no more than 10 submissions per hour per token).
4. **Looks up the org** by `public_request_token`. If the token isn't real, return 404.
5. **Creates a `pending_confirmation` order row** in the database with all the data.
6. **Triggers Resend to send a confirmation email** to the requester with a confirm-link.
7. **Returns** `{ id: <new-order-id>, trackUrl: <where-to-see-status> }` as JSON.

Step 3 — Back to the browser

The `fetch` call from step 1 receives that JSON. The React component:

- Calls `setSubmitted({ id, email, trackUrl })` — that flips an internal state variable.
- React re-renders, and the cart page disappears, replaced by a "Check your inbox" panel.
- The user clicks the confirm link in their email — that triggers a SECOND server-side flow that flips the order from `pending_confirmation` to `requested`, where it's visible to managers in the dashboard.

What that walkthrough teaches

Every user action is a chain: UI → JSON request → server validates → service does the DB work → JSON response → UI updates. Once you internalize that, every feature in the system follows the same shape. The trick to debugging or adding a new feature is figuring out which link of that chain is broken or needs to grow.

Chapter 7 — The database, the source of truth

The database is a Postgres instance hosted by Supabase. It contains 50+ tables. Here are the most important ones, grouped:

Core entities

Table	What it holds
<code>organizations</code>	One row per organization (your L4L workspace is one row). Holds name, terminology, public-link token.
<code>organization_members</code>	Joins users → organizations with a role (owner / admin / manager / staff / viewer).
<code>user_profiles</code>	Per-user data (name, avatar, etc.).
<code>warehouses</code>	A physical location (DC4/Roc, DC1/Fresno, etc.). Items belong to one warehouse.
<code>user_warehouse_assignments</code>	Which warehouses a staff/viewer is scoped to.
<code>charters</code>	A school site (e.g. "Roc Fresno"). "Deliver to which site" picks from these.

Inventory

Table	What it holds
<code>inventory_items</code>	Every item ever tracked. T-shirts, books, canopies, electronics, everything. ~50 columns. Notable booleans: <code>is_bundle</code> , <code>is_rental</code> .
<code>item_images</code>	One row per uploaded photo. Stores <code>storage_path</code> (where in Supabase Storage the file lives), <code>thumb_path</code> , <code>lqip</code> (blur preview).
<code>item_tags</code>	Many-to-many between items and tags.
<code>categories</code>	Per-org tags like Textbook, Apparel, Swag.
<code>locations</code>	Sub-warehouse locations (a specific room/shelf area).
<code>suppliers</code>	Who you buy from.

Movement / stock

Table	What it holds
<code>stock_movements</code>	An immutable ledger of every quantity change ever. Adjustments, transfers, receives, picks, etc. This is gold — never delete from it.
<code>stock_reservations</code>	"X units of item Y are held by reference Z right now." When you place an order or rent a canopy, rows go in here. When the order ships or the rental is returned, <code>released_at</code> gets set.

Workflow tables

Table	What it holds
<code>order_requests</code>	Header per order (requester, warehouse, status, dates).
<code>order_request_lines</code>	Items + quantities per order.
<code>rentals</code>	Header per rental (borrower, warehouse, dates, status). NEW in 0131.
<code>rental_lines</code>	Items + quantities per rental. NEW in 0131.
<code>purchase_orders</code> + <code>purchase_order_lines</code>	Purchase orders to suppliers.
<code>cycle_count_sessions</code> + <code>cycle_count_lines</code>	A counting audit ("count every item in DC4 today").
<code>bundles</code> + <code>bundle_components</code> + <code>bundle_distributions</code>	Kit / box assembly tracking.
<code>procedures</code> + <code>procedure_sections</code>	Standard operating procedures stored in-app.
<code>schedule_events</code>	Calendar entries.

Security + audit

Table	What it holds
<code>activity_logs</code>	Every privileged action: who did what, when, on which row. The audit trail.
<code>user_category_assignments</code>	Which categories a restricted viewer can see (NEW in 0128).
<code>notifications</code>	Persistent notification rows shown in the bell icon.
<code>notification_preferences</code>	Per-user opt-in/out per channel.

Other

- `tags`, `attachments`, `messages` (procedure comments), `mfa_recovery_codes`, etc.

The full SQL schema is split across **128 migration files** in `supabase/migrations/`. Each migration is one ordered change — `0001_init.sql` creates the original tables, `0131_rentals.sql` added the rentals tables yesterday. Migrations only run forward, never backward, which means the production database is always in a known state.

Important concept: the database is not just "a place to store data." Many of our business rules live in the database too — RLS policies, check constraints, foreign keys, helper functions (RPCs). That's by design. If a future bug in our app code tried to do something it shouldn't, the database itself would reject it.

Defense in depth.

Chapter 8 — How permissions work

There are five roles, ordered most-to-least privileged:

Role	What they can do
owner	Everything. Including billing, ownership transfer, deleting the workspace. One per org.
admin	Everything except billing + transfer.
manager	Approve orders, generate PDFs, edit team, see all warehouses. Most day-to-day power users.
staff	Operate the warehouse — pick orders, adjust stock, run cycle counts. Scoped to assigned warehouses.
viewer	Read-only. Can submit order requests but not approve anything. Can be category-restricted.

Permissions are defined in `packages/core/src/constants/permissions.ts`. The list of permissions includes things like `items:read`, `orders:approve`, `rentals:create`, `members:assign_categories`, etc. — about 40 of them.

Each role has a hardcoded list of which permissions it gets. The code uses `hasPermission(role, 'orders:approve')` thousands of times to decide whether to render a button, allow a server action, etc.

A critical concept: permissions are checked in THREE places for every sensitive action:

1. **UI level** — the button isn't rendered if the user doesn't have the permission. Cosmetic only — keeps the screen clean.
2. **Action level** — the server-action function calls `assertPermission(ctx, 'orders:approve')` before doing any DB work. If the user lacks the permission, throw an error.
3. **Database level (RLS)** — the database itself enforces a row-level rule via Postgres policies. Even if all the app code were bypassed, RLS would refuse.

The third layer — the database — is the only one that actually enforces. The first two are convenience and developer-error-prevention. **Always have all three.**

Category-restricted viewers

A new feature shipped yesterday lets you restrict individual viewers to specific categories (e.g., a parent volunteer who should only see Books, not Wellness Items). The data lives in `user_category_assignments`. The enforcement is in a Postgres function called `user_can_see_item_category()` which is called by RLS policies on `inventory_items` and `categories`.

If a viewer has zero rows in `user_category_assignments`, they see everything (back-compat). If they have one or more rows, they see only those categories AND no uncategorized items.

Chapter 9 — The major features, one section each

9.1 — Inventory items + books

The biggest feature. Lets you create, edit, search, photograph, and audit items.

Pages:

- `/dashboard/inventory` — list of items (filtered to non-books)
- `/dashboard/inventory/new` — create item form
- `/dashboard/inventory/[id]` — item detail (photos, history, edit, adjust stock)
- `/dashboard/books` — same as inventory but for books (`item_type = 'book'`)

How the list works:

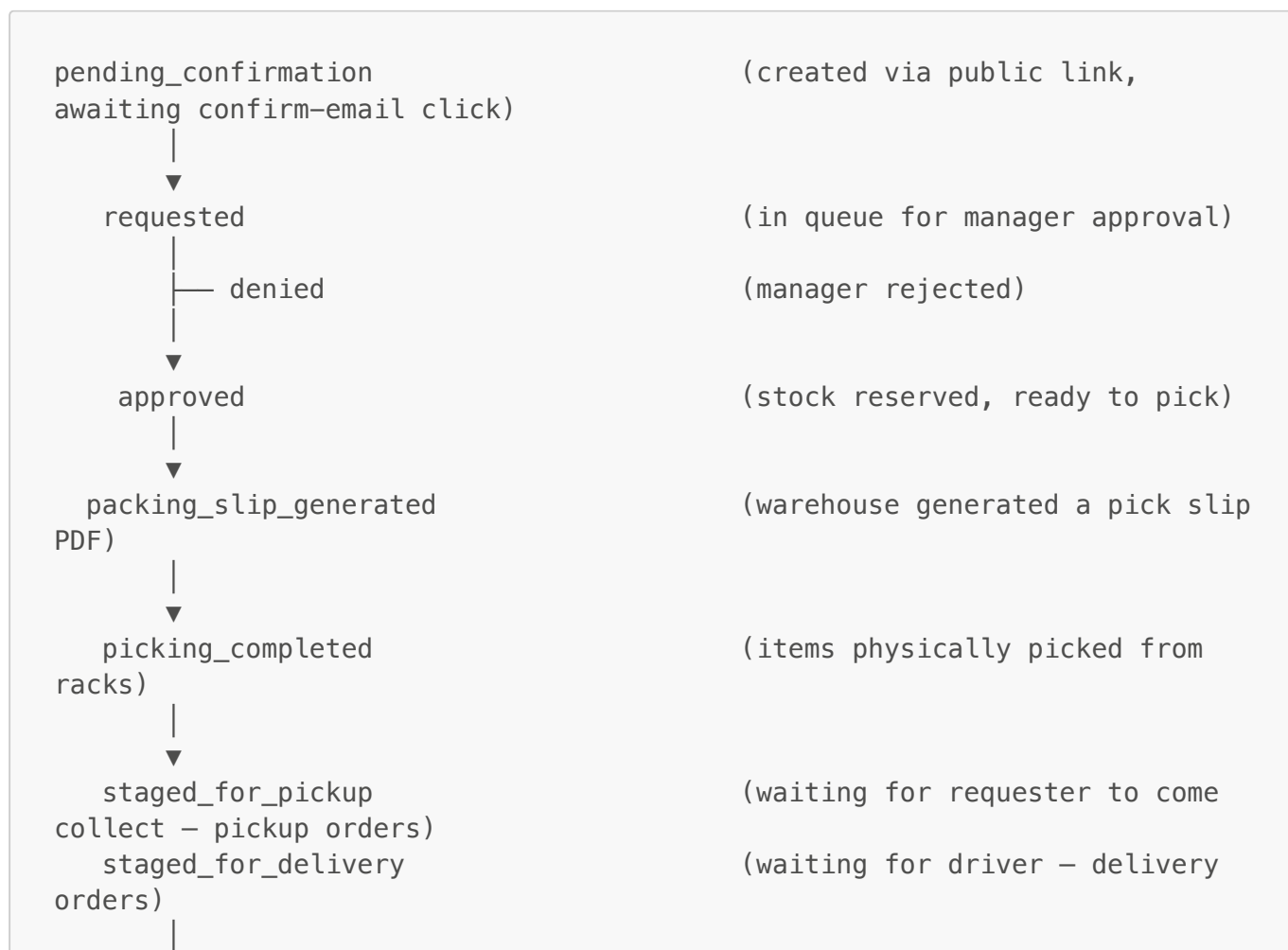
1. Server component runs `InventoryService.list(filters)` (`apps/web/src/server/services/inventory.ts`)
2. That returns rows from `inventory_items` with all the filters (warehouse, category, low-stock, etc.) applied
3. The page renders `<InventoryTable>` which is a fully-featured table component with sorting, pagination, bulk actions, etc.

How photos work — see Chapter 10.

9.2 — Orders (request → approve → fulfill → sign)

The most complex feature. Started as a public link for partners to request books; grew into a full ordering workflow.

The lifecycle:





Files involved:

- `packages/core/src/order-state-machine.ts` — defines legal transitions
- `apps/web/src/server/services/order-requests.ts` — every workflow action (approve, deny, generate-pick-slip, complete-picking, stage, mark-in-transit, etc.)
- `apps/web/src/app/(dashboard)/dashboard/orders/[id]/page.tsx` — the detail page
- `apps/web/src/components/orders/manager-actions-panel.tsx` — the buttons that drive the state machine
- `apps/web/src/components/orders/v2/` — the new aisle-catalog order picker (shipped May 2026)
- `apps/web/src/components/orders/public-v2/` — same picker, adapted for the public link

The signature flow has its own quirks:

- Public signature page at `/orders/sign/<token>` collects a hand-drawn signature on a canvas.
- "Someone else is receiving" toggle lets a different person sign on behalf of the original requester.
- Completion writes `signed_by_name + signed_by_email + signature_data_url` back to the row.

9.3 — Rentals (shipped 2026-05-19)

Internal checkout/return for circulating assets (canopies, supplies). No money — just accountability.

Two top-level tabs:

- **Activity** (default) — current and past rental transactions
- **Items** — the rental catalog (what CAN be rented)

Key design choice: rental items are a SEPARATE inventory class from regular items. The boolean flag `inventory_items.is_rental` partitions them. Every existing surface (orders picker, inventory list, AI search, reports) filters `is_rental = false` so a canopy never appears alongside a t-shirt.

Lifecycle:

```

out  → returned  (normal)
     → cancelled (never picked up)
  
```

Overdue is derived, not stored: `status='out' AND expected_return_at < now()`. No cron job needed.

Stock impact: when you rent a canopy, the system writes a row into the existing `stock_reservations` table with `reference_type='rental'`. The order picker subtracts that from available stock

automatically. When the rental is returned or cancelled, `released_at` is set and the canopy reappears as available.

Files: same shape as orders. `apps/web/src/server/services/rentals.ts`, `apps/web/src/components/rentals/`, `apps/web/src/app/(dashboard)/dashboard/rentals/`.

9.4 — Cycle counts + AI shelf scan

A cycle count is an audit: "go count every item on shelf 38-B and tell us what's there." Used to catch shrinkage and find lost stock.

The traditional flow:

1. Manager starts a count session (which warehouse, which items/categories).
2. Staff goes to the shelf with a phone, scans barcodes, types quantities.
3. The system compares counts to system stock — variance becomes a `stock_movement` adjustment.

The **AI shelf scan** (shipped May 2026) is a shortcut for books: take ONE photo of a shelf, Google Gemini reads the spines, returns ISBN matches with confidence scores. Counts above 0.85 confidence auto-populate the cycle count line.

Files: `apps/web/src/server/services/cycle-counts.ts`, `apps/web/src/lib/ai/shelf-scan.ts`, mobile screen at `apps/mobile/app/cycle-count/ai-scan/[id].tsx`.

9.5 — Reports + PDFs

Nine pre-built reports at `/dashboard/reports`:

- Inventory valuation (\$ value of everything)
- Stock movement summary (in/out trends)
- Reorder forecast (items at/below reorder point)
- Shrinkage & adjustments (negative movements last 30d)
- Supplier scorecard (on-time, lead time, fill rate)
- Velocity classes (ABC analysis — what moves most)
- Dead stock (on-hand but no out-movement)
- Bundle activity (distribution runs)
- Bundle shortages (components that ran short)

Every report has both a CSV and PDF export. The PDFs are generated server-side using `@react-pdf/renderer`. Item photos are embedded inline (with smart caching — see Chapter 11).

PDFs you may have seen:

- Inventory snapshot PDF (overview of everything)
- Per-report PDFs (any of the 9 above)
- Warehouse packing slip (when picking orders)
- Customer packing slip (what's in the box)
- Pick slip (instructions for the picker walking the racks)
- Cycle count PDF

9.6 — AI features

Two AI surfaces:

1. **AI assistant** at `/dashboard/ai` — natural-language chat. "How many size-XL Stronger Than Ever shirts do we have?" The assistant has tools (`apps/web/src/lib/ai/tools.ts`) that translate the question into database queries and synthesize an answer.
2. **AI shelf scan** — described above.

Both powered by Google Gemini 2.0 Flash via the standard Gemini SDK.

Chapter 10 — How images and PDFs work

This took a lot of polish work. The path is:

Upload (browser)

1. User picks a photo on the item form.
2. A Web Worker (`apps/web/src/lib/image-variants.worker.ts`) runs in the background: it resizes the photo to a 2048px master + a 200px thumbnail + a tiny 16×16 LQIP blur preview. Saves bandwidth on upload.
3. Three files are uploaded to Supabase Storage in the `item-images` bucket.
4. A row is inserted into `item_images` with the paths to those three files + the base64 LQIP string.

Display (browser, fast path)

1. Server component lists items via `InventoryService.list()`.
2. For each item shown, the server calls `ItemImagesService.primaryImagesWithThumbsForItems()` to get a `thumbUrl` + LQIP.
3. The card renders the LQIP blur as backdrop immediately, then crossfades to the real thumbnail when loaded.

Display (PDF embedding)

1. PDF generator gets a list of `itemIds`.
2. Calls `primaryImagesForPdfRendering(itemIds, 200)` — prefers `thumb_path`, falls back to a transformed master, falls back to `custom_fields.thumbnail_url` (for bulk-imported books).
3. URLs are cached via `unstable_cache` (25-day TTL) so subsequent PDFs are near-instant.
4. Image bytes are then fetched and converted to base64 data URIs (also cached) and embedded directly in the PDF — no per-image HTTP at render time.

Key reason we don't render giant master images: a 2048×2048 photo is ~1-2MB. 272 items × 1MB = 270MB of bandwidth on one page load. Using 200px thumbnails (~30KB each) drops that 30× to ~8MB.

Chapter 11 — How real-time updates work

When you create an item on your phone, the inventory list on your computer refreshes in ~250ms. Here's how:

1. **Mobile app inserts a row** into `inventory_items` via a server action.

2. **Postgres logical replication** picks up the change (Postgres has a built-in feature called WAL — write-ahead logging).
3. **Supabase Realtime relay** reads from WAL and broadcasts the change over a WebSocket.
4. **The browser dashboard is subscribed** via `InventoryRealtime` (`apps/web/src/components/realtime/inventory-realtime.tsx`). The subscription is set up once when the dashboard loads.
5. **On receiving the event**, the subscription calls `router.refresh()` — Next.js's "re-run the server components and re-render."
6. **The server component re-fetches** the now-fresh inventory list and ships an updated HTML chunk.
7. **React diffs it against the current DOM** and only updates the changed rows.

The whole loop takes ~200-400ms in practice. Tightened from a 750ms trailing debounce to a 250ms leading-edge throttle in the May 2026 polish round.

A concept worth remembering: RLS applies to realtime subscriptions just like it does to direct queries. The server only broadcasts row events to subscribers whose RLS policies would let them see those rows. So a warehouse-scoped staff user never receives events for items in another warehouse — without us writing any extra code.

Chapter 12 — Caching, the secret weapon

Most pages would be slow if they re-queried Supabase on every visit. They don't, because of two cache layers:

Layer 1 — `unstable_cache` (Vercel Data Cache)

A Next.js feature that wraps a function. The first call hits the database; subsequent calls within a TTL window return the cached result.

We use it for:

- Signed image URLs (25-day TTL — same URL for the same path, until rotated)
- Image bytes as base64 data URIs (25-day TTL — used by PDFs)
- The orders/new catalog payload (30-second TTL — items don't change that fast)

Layer 2 — Tag-based invalidation

Caches are tagged. When something changes, the action calls `updateTag('orders-new-v2-catalog')` to purge all entries with that tag. Next visit recomputes.

The crucial trick: for per-user data, the cache key **MUST** include something user-specific. Otherwise a manager's full-catalog payload could be served to a restricted viewer who shouldn't see all of it. We learned this the hard way on the category-visibility feature — fixed via a hash of the user's accessible categories baked into the cache key.

Chapter 13 — Anti-bot + security tricks worth knowing

1. **Honeypot fields.** The public order form has a hidden `<input name="website">` positioned off-screen with `aria-hidden`. Real users never see it. Naive bots fill every field. If that field is non-empty on submit, the server silently returns 200 without saving — the bot has no signal it failed.
 2. **Rate limits.** Public endpoints cap submissions per IP + per email + per org. Closed-mode (DB outage = deny) so a database hiccup doesn't unlock unlimited submissions.
 3. **Signed URLs for storage.** Item photos are private — they have signed URLs with a 30-day expiry. The signature is computed by Supabase from a secret. No one can guess a working URL.
 4. **Defense-in-depth on RLS.** Every read-sensitive surface checks permissions at the page, action, AND database layer. If any one is bypassed, the other two still hold.
 5. **security definer Postgres functions.** A function declared `security definer` runs with the rights of its creator (typically `postgres`), bypassing RLS. We use this carefully — e.g., the `confirm_order_signature` RPC needs to update an order row owned by another user. Anywhere we use `security definer`, the function itself enforces its own checks.
 6. **Never commit .env files.** Hard rule — we burned ourselves once with a credential leak. Now the codebase has multiple `.gitignore` patterns and a hook that blocks any `.env*` write.
-

Chapter 14 — The patterns we use over and over

Once you see these, you'll recognize them everywhere.

Pattern 1 — page → action → service → database

Every feature follows this four-layer flow. Top to bottom is "where the work happens." Bottom to top is "where the data flows back."

Pattern 2 — zod schemas guard every input

Any data crossing a trust boundary (browser → server, mobile → server) gets parsed by a zod schema (`packages/core/src/schemas/`). If the parse fails, the server returns a `validation_error` and refuses to do work.

Pattern 3 — service class per domain

`InventoryService`, `OrderRequestsService`, `RentalsService`, `CycleCountsService`, etc. Each owns its tables and exposes methods. Other code goes through the service — nothing writes to `inventory_items` directly except `InventoryService`.

Pattern 4 — Server Action returns `{ ok: true, data } | { ok: false, error }`

This shape is enforced via the `ActionResult<T>` type. Caller writes `if (!res.ok) toast.error(...)`. Type-safe and uniform.

Pattern 5 — Migration-first DB changes

Database changes ALWAYS go through a numbered migration file. Never edit existing migrations. Never run ad-hoc SQL on prod without a migration committed. This is the only way to keep prod and dev in sync.

Pattern 6 — Reservations for "soft locks"

When you place an order, the items are written to `stock_reservations` not subtracted from `quantity_on_hand`. Same for rentals. Available-to-promise = `quantity_on_hand - SUM(stock_reservations.quantity WHERE released_at IS NULL)`. Releasing a reservation is setting `released_at = now()`, never deleting the row. Audit trail preserved.

Pattern 7 — Soft deletes

We never `DELETE FROM inventory_items WHERE id = ....` Instead we set `deleted_at = now()`. Every read filters `WHERE deleted_at IS NULL`. Restore-by-toggle is possible. Audit trail preserved.

Pattern 8 — Audit log writes are fire-and-forget

After a privileged action, we call `void audit({...}, ctx)` without awaiting. If the audit insert fails (rare), the user-facing action still succeeded. Audit logs are eventual-consistent.

Pattern 9 — Leading-edge throttle for realtime refreshes

First event in a window triggers `router.refresh()` immediately for instant feedback; subsequent events within the throttle window get coalesced into one trailing refresh. Better than trailing debounce which makes every event wait.

Pattern 10 — Subagents for parallel work

When implementing a feature, we dispatch sub-agents for independent tasks (e.g. "build the UI" in parallel with "write the service"). Each sub-agent gets a focused brief + verification gate. This is what lets us ship features faster than we could solo.

Chapter 15 — Things to write down + tricks to remember

A starter list of concepts that recur. Worth a sticky note above your monitor.

1. **The four-layer flow** — every action: page → server action → service → database. If you're lost, find which layer you're in.
2. **RLS is the security floor**. Application code is convenience. The database itself enforces who sees what.
3. **Same-command RLS policies are OR'd, not AND'd**. This bit us on the category-visibility feature. If you add a new SELECT policy to a table that already has one, BOTH must permit the row — they're combined with OR. The fix is usually to REPLACE the existing policy with one that AND's the new condition in.
4. **updateTag after writes**. When you change something that's cached, call `updateTag('the-tag')` from a server action. Otherwise the dashboard serves stale data.

5. **Migrations only go forward.** Never edit a migration that's been applied. To "undo," write a new migration that does the opposite.
 6. **security definer functions skip RLS.** Use them only when you know why you need to, and make the function check its own preconditions.
 7. **Server Actions auto-refresh the page tree.** If you call a server action from a page that should "stay mounted with success state," use a route handler (`fetch('/api/...')`) instead. Auto-refresh would unmount your success panel.
 8. **Always run `pnpm typecheck` & `pnpm lint` after changes.** Vitest passes don't mean CI passes. TypeScript-strict catches errors vitest doesn't.
 9. **Public surfaces need rate limits + honeypot.** Anything accessible without auth is a target. Both protections together are cheap.
 10. **Soft-delete > hard-delete.** Always. Setting `deleted_at` preserves the audit trail and lets you restore.
 11. **Item photos: 30KB thumbnails, not 2MB masters.** PDFs and lists should always use `thumb_path`. The master is only for the lightbox view on click.
 12. **Stock reservations release on return — they don't delete.** `released_at` is the soft-release marker. Available = NOT NULL → in effect, NULL → released.
 13. **When a feature uses an existing table, prefer adding a column or a `reference_type` rather than a new table.** Rentals reuse `stock_reservations` with `reference_type='rental'`. Orders reuse it with `reference_type='order_request'`. Saves us from threading "available math" through three different code paths.
 14. **Realtime requires the table to be in `supabase_realtime` publication.** If you add a new table and subscribe to it, but events never arrive — you forgot the publication membership migration (see `0117_realtime_order_requests.sql`, `0132_realtime_rentals.sql`).
 15. **Cache keys must include user-scoped fragments for per-user data.** If two users with different visibility hit the same cached key, the wrong one gets the other's payload. The fix is to bake a hash of "who is the user and what can they see" into the key.
-

Chapter 16 — How to keep learning

Some practical suggestions for getting more fluent over time.

1. **After every feature ships, ask for a "what just happened" walkthrough.** A memory rule is now in place: every new feature gets a plain-English explainer with 1-2 concepts to remember. Take notes when those land.
2. **Open one file a week and read it.** Pick a file from a feature you use, open it in GitHub or VS Code, and skim it. You don't need to understand every line — just notice the shape. Comments are written for someone new.

3. Read the migrations folder. Each numbered file in `supabase/migrations/` is a self-contained change to the database. They have detailed comments explaining WHY each change was made. Reading them in order is like reading a history of the product.

4. Follow a feature end-to-end once. Pick the simplest feature (e.g. "duplicate item") and trace it from the button click to the database write. The chain is:

- `apps/web/src/components/inventory/duplicate-item-dialog.tsx` — the dialog
- `apps/web/src/server/actions/duplicate-item.ts` — the action
- `apps/web/src/server/services/inventory.ts` (the `duplicateItem` method) — the service
- `supabase/migrations/0125_duplicate_inventory_item.sql` — the RPC that does the atomic copy

5. Use `git blame` in GitHub. Click the "Blame" button on any file. Every line is tagged with the commit + commit message that introduced it. You can read the WHY for any line of code in 30 seconds.

6. When you see a word you don't know, ask. There's no dumb question. The terminology has a finite size — about 100 terms total — and you'll absorb them in a couple months of repeated exposure.

Glossary

Term	Plain meaning
API	A defined way for one program to talk to another. We have "API routes" for the mobile app to talk to the server.
bucket	A folder in Supabase Storage. We have <code>item-images</code> for photos, etc.
cache	Saved-for-later results. Saves time on repeated work.
CSP	Content Security Policy — a browser security header that tells the browser what's allowed.
cron	A scheduled task that runs on a recurring schedule. Our weekly digest runs via Vercel cron.
data URI	A way of embedding image bytes directly into HTML/PDF as a base64 string instead of a separate URL.
debounce	"Wait until events stop, then do the thing once." Used to collapse bursts.
deploy	Push code to production. We deploy on every commit to <code>main</code> .
FK / foreign key	A database column that points to a row in another table. Enforces referential integrity.
hook	A small reusable React function that holds state or behavior (e.g. <code>useState</code> , <code>useRouter</code>).
LQIP	Low-quality image placeholder — a tiny blurry preview shown while the real photo loads.
migration	A numbered SQL file that changes the database schema. Migrations only go forward.

Term	Plain meaning
<i>monorepo</i>	One git repo holding multiple apps + shared packages. Ours has web + mobile + 3 packages.
<i>RLS</i>	Row-Level Security — Postgres feature that lets us write security rules directly on tables.
<i>RPC</i>	Remote Procedure Call — in our context, a stored function in Postgres that we call from app code (<code>supabase.rpc('confirm_order_signature', {...})</code>).
<i>RSC</i>	React Server Component — a React component that runs only on the server. Most of our pages are RSCs.
<i>Server Action</i>	A function marked 'use server' that the browser can call as if it were a normal function but it actually runs on the server. Handles form submits.
<i>signed URL</i>	A URL with an embedded cryptographic signature that proves the holder is authorized. Used for private file access.
<i>SKU</i>	Stock-keeping unit. The internal code for an item. Often human-readable like <code>SP-RUQPL-LJC</code> .
<i>throttle</i>	"At most once every X ms." Different from debounce — throttle still fires on the first event.
<i>TTL</i>	Time to live. How long a cache entry stays valid.
<i>ULID/UUID</i>	Universally unique identifier. The id format for every row in our database (e.g. <code>00000000-0000-0000-0000-000000000001</code>).
<i>webhook</i>	An endpoint that an external service calls when something happens. We don't currently use webhooks.
<i>zod</i>	A JavaScript library for runtime input validation. We use it for every form + every API endpoint.

Appendix A — File map cheat sheet

If you're hunting for where something lives, this is the fastest reference.

What you're looking for	Where it lives
The thing rendered when I click X	<code>apps/web/src/app/.../page.tsx</code> (or a component imported by it)
The "Save" button's actual code	<code>apps/web/src/server/actions/...</code>
The business logic for X	<code>apps/web/src/server/services/X.ts</code>
What columns the X table has	<code>supabase/migrations/00XX_X.sql</code>

What you're looking for	Where it lives
Type definitions / shared shapes	<code>packages/core/src/schemas/</code> or <code>packages/core/src/types/</code>
Permission rules	<code>packages/core/src/constants/permissions.ts</code>
Email templates	<code>apps/web/src/lib/email/</code>
AI tool definitions	<code>apps/web/src/lib/ai/tools.ts</code>
PDF rendering	<code>apps/web/src/lib/pdf/</code>
Mobile screens	<code>apps/mobile/app/</code>
Tests	<code>apps/web/src/**/*.test.ts</code>
Memory + workflow rules	<code>~/.claude/projects/.../memory/</code>
Specs and plans for shipped features	<code>docs/superpowers/specs/</code> and <code>docs/superpowers/plans/</code>

Appendix B — Useful commands

When you're at your computer in the project folder:

Command	What it does
<code>pnpm install</code>	Install all dependencies
<code>pnpm -F @stockpilot/web dev</code>	Start the web app locally on <code>localhost:3000</code>
<code>pnpm -F @stockpilot/web test</code>	Run the test suite
<code>pnpm -F @stockpilot/web typecheck</code>	Check for TypeScript errors
<code>pnpm -F @stockpilot/web lint</code>	Check for code-quality warnings
<code>git status</code>	See what files you've changed
<code>git log --oneline -20</code>	See the last 20 commits

Appendix C — Recently shipped features (rough timeline)

For context — these are the major shipped features in the last ~6 weeks:

Approximate date	Feature
April 2026	Public order link (<code>/r/[token]</code>) — initial version
Early May 2026	Internal-company pivot (away from public SaaS)
May 2026	Mobile native drawer nav
May 2026	AI Shelf Scan

Approximate date	Feature
May 2026	Email deliverability + Resend setup (SPF/DKIM/DMARC)
May 2026	PDF reports for all 9 reports with item photos
Mid-May 2026	Duplicate item button
Mid-May 2026	Orders v2 (aisle-catalog picker)
Mid-May 2026	Public link v2 (same aisle picker for <code>/r/[token]</code>)
Late May 2026	"Someone else is receiving" toggle on signatures
Late May 2026	Viewer category visibility (restrict viewers to certain categories)
2026-05-19	Rentals feature (this one — circulating-asset checkout/return)

End of guide. Print at letter or A4, double-sided is fine. Keep a copy near your desk and one in the warehouse office. When something doesn't match what's described here, that's a sign the doc needs updating — let me know and we'll keep it current.